Continuous Integration

Run locally. Push. Read the result. Repair.

Gregory S. DeLozier, PhD

Tests need to get run

A working change can still break another behavior.

Run the checks on each pushed version.

Keep the result attached to the version that was tested.

One app, three testing methods

Method	Exercises	Passing baseline
Direct calls with pytest	Calculator logic	27 tests
NiceGUI simulation	Button handlers and display	12 tests
behave + WebDriver	Running app in Chrome	12 scenarios

GUI: graphical user interface.

Testing reach makes compartmentalization useful

Without direct access, testing the math becomes clicking web buttons.

Check	Reach it through
Arithmetic	`Calculator` methods
Button wiring and display updates	NiceGUI simulation
Real browser interaction	behave + WebDriver

Check each part directly, then check its connections.

Extract the student starting point

Extract **webcalc.zip** into a fresh working location.

The archive includes `.github/` and `.gitignore`.

```
webcalc/
  main.py                      calculator.py
  test_main.py                 test_calculator.py
  requirements-test.txt        run_browser_tests.py
  features/
  .github/workflows/webcalc-tests.yml
```

Open a terminal in this extracted folder.

Create a local environment

Use Python 3.12 to match the Actions jobs.

macOS / Linux:

```
python3.12 -m venv .venv
source .venv/bin/activate
```

Windows PowerShell:

```
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
```

After activation, use `python` on either platform.

Run the app locally

```
python -m pip install -r requirements-test.txt
python main.py
```

Open `http://localhost:8080`.

Click `2`, `+`, `3`, `=`. Expected display: **5**.

Stop the app with **Ctrl+C** before the browser runner.

Run the unit tests first

```
python -m pytest -v test_calculator.py
```

Expected: **27 passed**.

The tests call `Calculator` directly.

No NiceGUI page or browser is involved.

Run the simulated GUI tests

```
python -m pytest -v test_main.py
```

Expected: **12 passed**.

NiceGUI simulates clicks on the real page's Python elements.

The checks read the display label, not a numbered button.

Run the real browser tests

```
python run_browser_tests.py
```

Expected: **12 scenarios, 51 steps passed**.

Runner action	Result
Start WebCalc and wait	App responds on port 8080
Run behave	Headless Chrome exercises the app
Finish	App process stops

Save the passing baseline

```
git init -b main
git add .
git status --short
```

Verify the app, tests, and workflow are staged.

```
git commit -m "Add WebCalc with three layers of tests"
```

All three test layers have passed locally.

Create webcalc on GitHub

With GitHub command-line interface (CLI) installed:

```
gh auth login
gh repo create webcalc --private --source=. --remote=origin --push
```

Expected: a new **webcalc** repository containing the baseline.

Use `--public` for a public class repository.

Alternative: create it on the website

Create an empty repository named **webcalc** on GitHub.

Leave README, license, and gitignore initialization unchecked.

Replace `YOUR_USERNAME`, then run:

```
git remote add origin https://github.com/YOUR_USERNAME/webcalc.git
git push -u origin main
```

Use this route or the CLI creation route, not both.

The workflow lives with the code

File: `.github/workflows/webcalc-tests.yml`

```
name: WebCalc tests
on:
  push:
  pull_request:
  workflow_dispatch:
```

Pushes start runs. The manual trigger supports reruns on demand.

Three jobs from one matrix

```
strategy:
  fail-fast: false
  matrix:
    layer: [unit, simulated-gui, browser]
```

Each layer gets its own runner and result.

A unit failure does not cancel the other two jobs.

The same commands, on another machine

Job	Test command
unit	`python -m pytest -v test_calculator.py`
simulated-gui	`python -m pytest -v test_main.py`
browser	`python run_browser_tests.py`

The workflow adds report options and saves artifacts.

No development server needs to be left running.

Inspect the first Actions run

Open **Actions** > **WebCalc tests** > **the first push**.

Job	Expected
WebCalc / unit	27 passed
WebCalc / simulated-gui	12 passed
WebCalc / browser	12 scenarios, 51 steps passed

Expand each test step and read its output.

Break one expectation

File: `test_calculator.py`, function `test_addition`.

Change the final assertion to:

```
assert calc.display == '6'
```

```
python -m pytest -v test_calculator.py -k test_addition
```

Expected: **1 failed**. Actual `5`; deliberately expected `6`.

Push the known failure

```
git add test_calculator.py
git commit -m "Demonstrate a failing addition check"
git push
```

Open the new Actions run.

unit	simulated-gui	browser
Fail	Pass	Pass

Read the failed assertion

Open **WebCalc / unit > Test calculator logic**.

Expected comparison:

```
actual display:    '5'
expected display:  '6'
```

The test is wrong in this exercise. Repair the expectation.

Repair it locally

Restore `test_addition`:

```
assert calc.display == '5'
```

Run all three layers:

```
python -m pytest -v test_calculator.py
python -m pytest -v test_main.py
python run_browser_tests.py
```

Expected: **27 tests, 12 tests, 12 scenarios pass**.

Push the repair and confirm green

```
git add test_calculator.py
git commit -m "Restore the correct addition expectation"
git push
```

Expected: **all three jobs pass on the new run**.

The earlier failed run remains attached to the earlier commit.

Keep the feedback useful

Run locally before pushing.

Inspect failures and repair the cause.

Check which revision produced the result.

Use automation to make the shared work easier to trust.